FIAP | DSA

Aula 4: Estruturas Lineares, Ordenação e Desempenho

Lista, Pilha, Fila, algoritmos de ordenação, Big-O e otimização

Prof. Dr. Erick Toshio Yamamoto

Objetivos da aula

Da organização dos dados à escolha consciente do algoritmo.

1. Compreender lista, pilha e fila e suas operações fundamentais.
2. Relacionar estruturas lineares com ponteiros, memória e recursividade.
3. Implementar e comparar Bubble, Selection, Insertion, Merge e Quick Sort.
4. Analisar melhor, médio e pior caso com notação Big-O.
5. Medir tempo real e discutir otimização de software e hardware.

Mapa da aula

A estrutura de dados define como o algoritmo enxerga e movimenta os dados.



Aula anterior: recursividade e pilha de chamadas → **Agora: estruturas lineares + algoritmos de ordenação**

Revisão rápida: struct + ponteiro + malloc

A lista ligada nasce exatamente da combinação destes três conceitos.

```c
typedef struct No {
    int valor;
    struct No *proximo;
} No;

No *novo = malloc(sizeof(No));
novo->valor = 10;
novo->proximo = NULL;
```

STRUCT

Agrupa o valor e o endereço do próximo nó.

MALLOC

Cria o nó dinamicamente durante a execução.

PONTEIRO

Guarda “onde” o próximo nó está na memória.

->

Acessa campos quando temos um ponteiro para a struct.

Esses conceitos formam a base das estruturas dinâmicas

O que são estruturas lineares?

Elementos organizados em uma sequência lógica.

Vetor

Acesso por índice

Lista

Nós conectados

Pilha

LIFO

Fila

FIFO

A diferença principal não é apenas “onde o dado fica”, mas quais operações queremos tornar rápidas.

Lista simplesmente ligada

Cada nó guarda um valor e um ponteiro para o próximo nó.



Vantagem

Inserção no início pode ser feita em tempo constante.

Custo

Para acessar o 3º nó, precisamos caminhar pelos anteriores.

Ref. visual: Wikimedia Commons – Singly-linked-list.svg (domínio público)

Operações em lista: inserir e percorrer

A posição da operação muda a complexidade.

```c
No *inserirInicio(No *inicio, int valor) {
    No *novo = malloc(sizeof(No));
    if (novo == NULL) return inicio;

    novo->valor = valor;
    novo->proximo = inicio;
    return novo;
}
```

Inserir no início: O(1)

```c
void imprimir(No *inicio) {
    No *atual = inicio;

    while (atual != NULL) {
        printf("%d ", atual->valor);
        atual = atual->proximo;
    }
}
```

Percorrer: O(n)

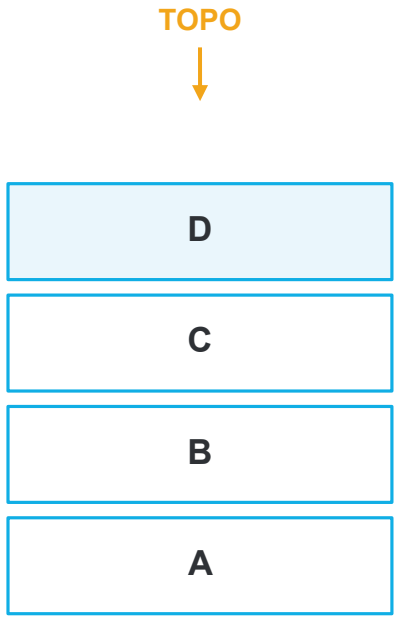
Lista ligada: qual operação custa mais?

Operação	Complexidade
Acessar o primeiro nó	O(1)
Inserir no início	O(1)
Percorrer todos os nós	O(n)
Buscar um valor	O(n)
Acessar a posição i	O(n)
Inserir no fim sem ponteiro “fim”	O(n)

Pergunta-chave: “preciso percorrer a estrutura antes de fazer a operação?”

Pilha — LIFO

Last In, First Out: o último a entrar é o primeiro a sair.



PUSH

Insere no topo da pilha.

PEEK / TOP

Consulta o topo sem remover.

POP

Remove o elemento do topo.

Aplicações: Ctrl+Z, histórico, avaliação de expressões, compiladores e chamadas de função.

Ref. visual: Wikimedia Commons – QUEUE VS STACK.svg (CC BY-SA 4.0)

Implementando pilha com lista ligada

Topo = ponteiro para o primeiro nó.

```
No *push(No *topo, int valor) {
    No *novo = malloc(sizeof(No));
    if (novo == NULL) return topo;
    novo->valor = valor;
    novo->proximo = topo;
    return novo;
}
```

push = O(1)

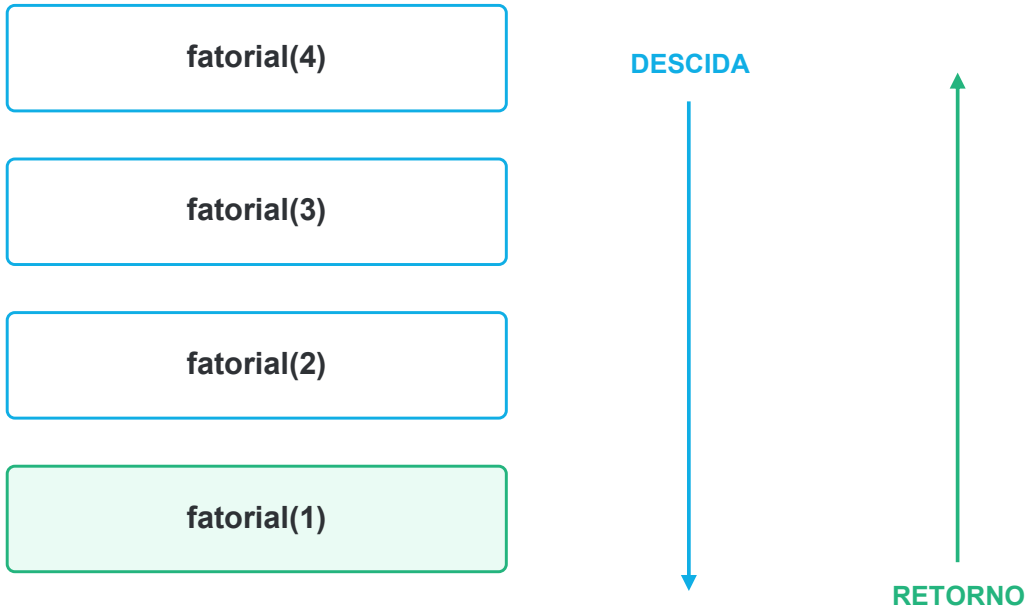
```
No *pop(No *topo) {
    if (topo == NULL) return NULL;

    No *remover = topo;
    topo = topo->proximo;
    free(remover);
    return topo;
}
```

pop = O(1)

Conexão com recursividade: a pilha de chamadas

A recursão da aula anterior já estava usando uma pilha.



Por que isso importa?

Uma recursão profunda consome memória da pilha; uma recursão sem caso base pode causar stack overflow.

Conexão com estrutura de dados

LIFO explica por que a última chamada feita é a primeira a retornar.

Fila — FIFO

First In, First Out: o primeiro a entrar é o primeiro a sair.



ENQUEUE

Insere no final da fila.

DEQUEUE

Remove do início da fila.

Aplicações: fila de impressão, requisições, buffers, redes e escalonamento.

Ref. visual: Wikimedia Commons – QUEUE VS STACK.svg (CC BY-SA 4.0)

Fila eficiente: ponteiros para início e fim

Com os dois ponteiros, inserir e remover podem ser O(1).

```c
typedef struct {
    No *inicio;
    No *fim;
} Fila;

void enfileirar(Fila *f, int valor) {
    No *novo = malloc(sizeof(No));
    novo->valor = valor;
    novo->proximo = NULL;
    if (f->fim == NULL) f->inicio = novo;
    else f->fim->proximo = novo;
    f->fim = novo;
}
```

```c
int desenfileirar(Fila *f) {
    if (f->inicio == NULL) return -1;

    No *remover = f->inicio;
    int valor = remover->valor;
    f->inicio = f->inicio->proximo;

    if (f->inicio == NULL)
        f->fim = NULL;

    free(remover);
    return valor;
}
```

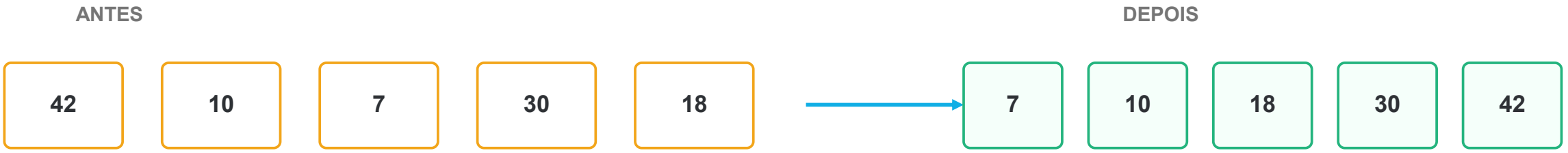
Lista, pilha e fila: mesma base, regras diferentes

Estrutura	Regra de acesso	Inserção típica	Remoção típica	Uso clássico
Lista	Sequencial	O(1) ou O(n)	O(1) ou O(n)	Coleções dinâmicas
Pilha	LIFO	O(1)	O(1)	Recursão / histórico
Fila	FIFO	O(1)	O(1)	Processamento em ordem

Estrutura adequada = menos trabalho desnecessário.

Agora os dados estão armazenados. Como ordená-los?

Ordenação transforma a organização dos dados e pode acelerar etapas posteriores.



Critério 1

Tempo: quantas comparações e movimentações?

Critério 2

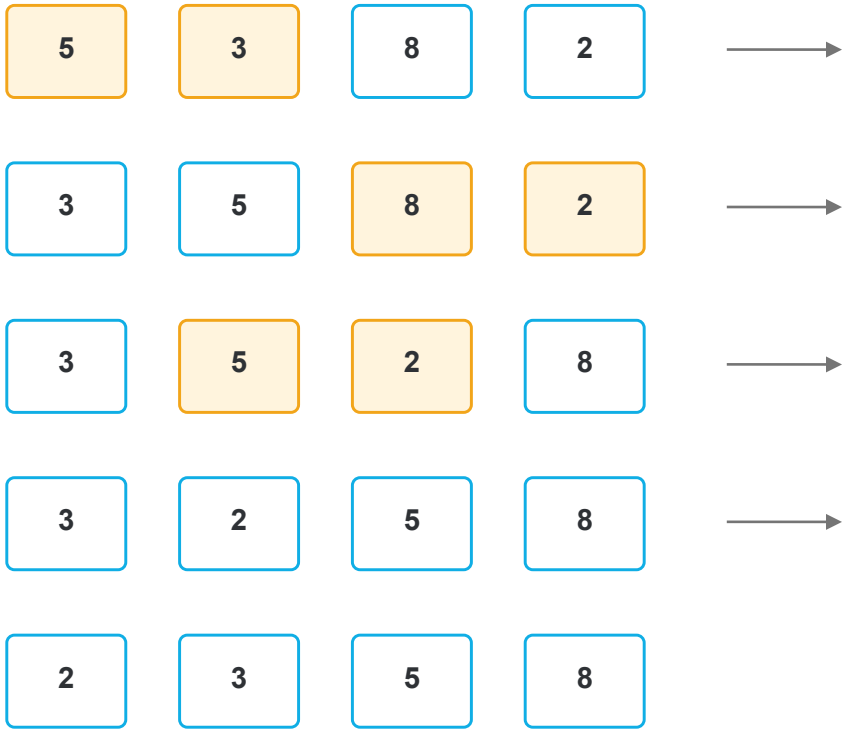
Memória: o algoritmo precisa de espaço extra?

Critério 3

Entrada: pequena, grande, quase ordenada ou aleatória?

Bubble Sort — comparar vizinhos e trocar

O maior elemento “borbulha” para o final a cada passagem.



Ideia

Compare pares adjacentes; troque quando estiverem fora de ordem.

Complexidade

Clássico: O(n²). Otimizado: melhor caso O(n).

Pior caso

Muitas passagens e muitas trocas.

Ref. visual: Wikimedia Commons – Bubble sort animation.gif (CC BY-SA 2.5)

Bubble Sort em C e de onde vem O(n²)

```c
void bubbleSort(int v[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (v[j] > v[j + 1]) {
                int temp = v[j];
                v[j] = v[j + 1];
                v[j + 1] = temp;
            }
        }
    }
}
```

$$(n-1) + (n-2) + \ldots + 1$$

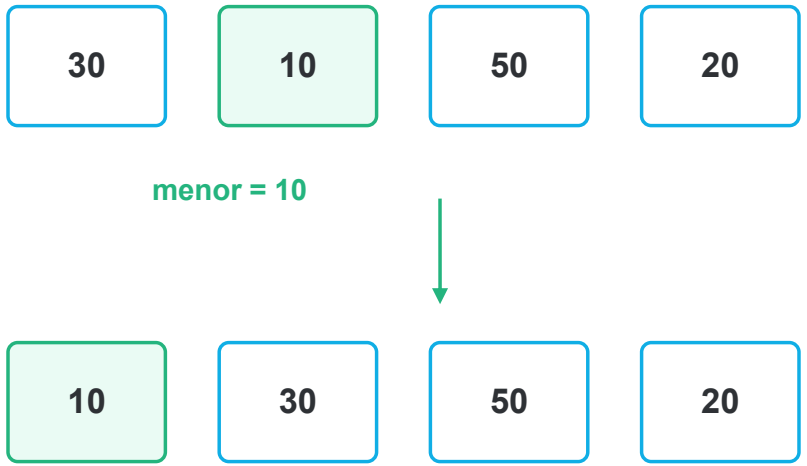
$$= n(n-1) / 2$$

$$= (n^2 - n) / 2$$

Termo dominante: $n^2 \rightarrow O(n^2)$

Selection Sort — selecionar o menor e posicionar

Menos trocas que Bubble, mas ainda muitas comparações.



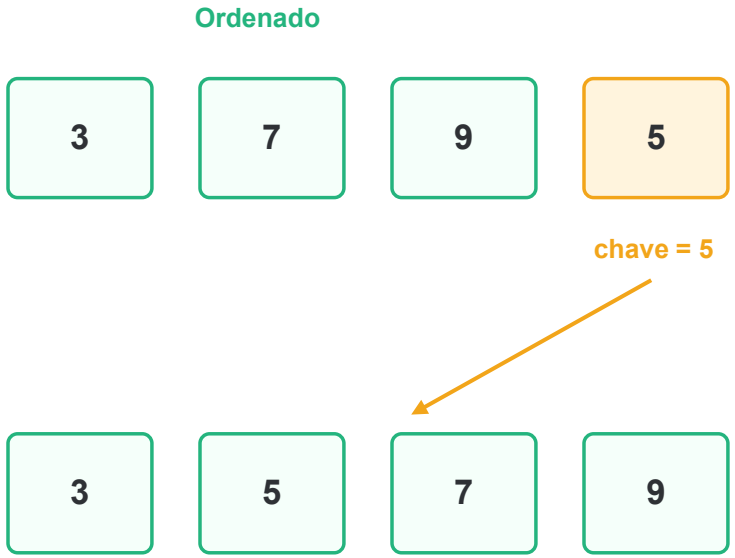

```
for (int i = 0; i < n - 1; i++) {
    int menor = i;
    for (int j = i + 1; j < n; j++)
        if (v[j] < v[menor]) menor = j;
    // troca v[i] com v[menor]
}
```

Comparações ≈ n(n−1)/2 → O(n²)

Ref. visual: Wikimedia Commons – Selection sort animation.gif (domínio público)

Insertion Sort — como organizar cartas na mão

Constrói uma região ordenada e insere cada novo elemento no lugar certo.



Melhor caso

Dados já/quase ordenados: O(n).

Boa escolha

Vetores pequenos e quase ordenados.

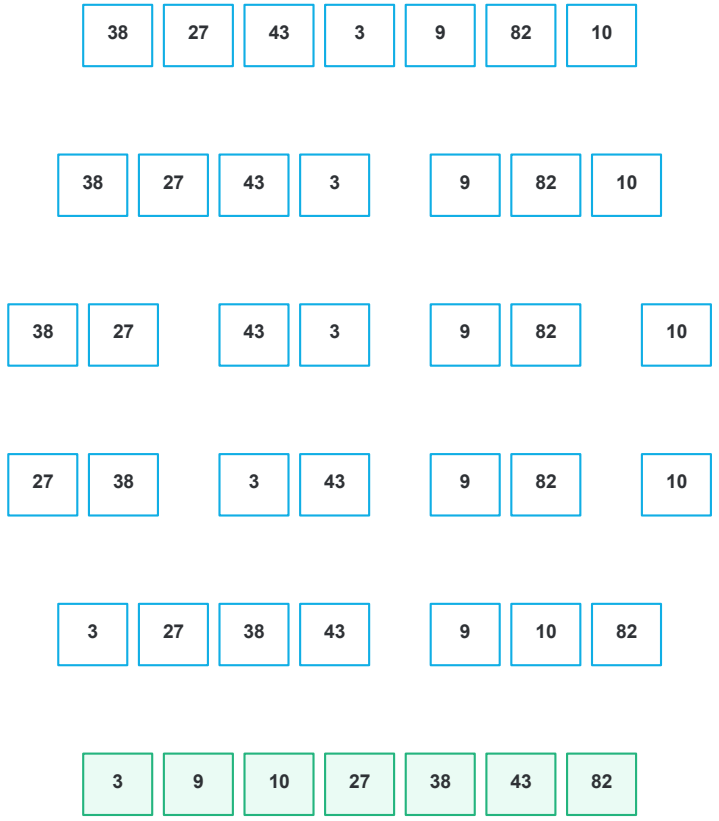
Pior caso

Ordem inversa: O(n²).

Ref. visual: Wikimedia Commons – Insertion-sort-example.gif (CC BY-SA 3.0)

Merge Sort — dividir, ordenar e combinar

Recursividade aplicada a um algoritmo de ordenação previsível.



Recorrência

$T(n)=2T(n/2)+O(n)$

Complexidade

Melhor = médio = pior = O(n log n)

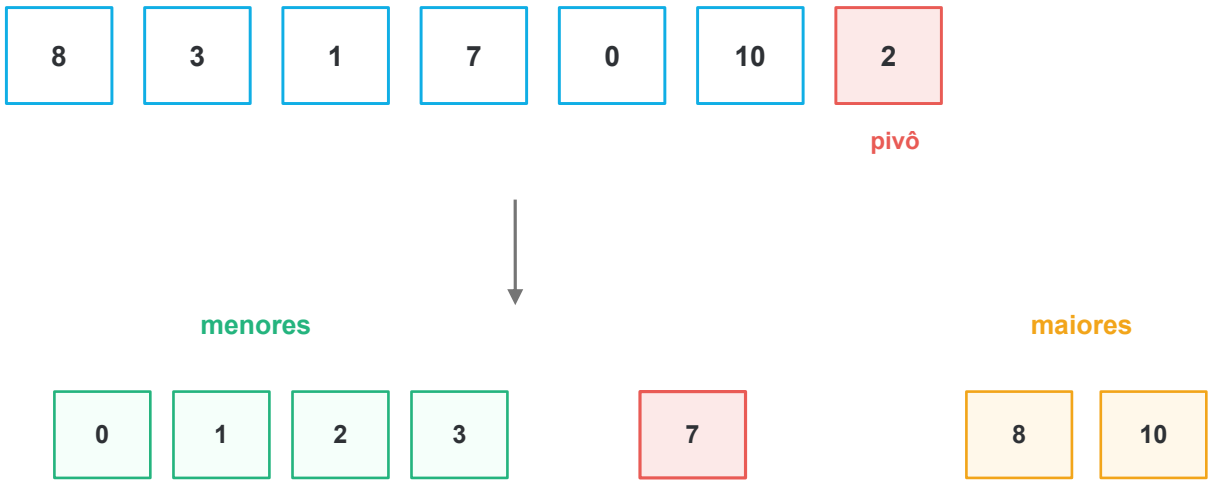
Memória

Em arrays, normalmente usa O(n) auxiliar.

Ref. visual: Wikimedia Commons – Merge sort animation.gif (CC BY-SA 2.5)

Quick Sort — particionar em torno de um pivô

O pivô separa valores menores e maiores; depois o processo é recursivo.



Caso médio

Partições equilibradas → O(n log n).

Pior caso

Partições muito desequilibradas → O(n²).

Na prática

Boa localidade em arrays e baixo custo auxiliar em versões in-place.

Ref. visual: Wikimedia Commons – Sorting quicksort anim.gif (imagem destacada)

Quick Sort em C — estrutura recursiva

A função particionar é o coração do algoritmo.

```
void quickSort(int v[], int inicio, int fim) {
    if (inicio < fim) {
        int p = particionar(v, inicio, fim);

        quickSort(v, inicio, p - 1);
        quickSort(v, p + 1, fim);
    }
}
```

1. Escolher pivô

Ex.: último elemento, mediana ou pivô aleatório.

2. Particionar

Menores de um lado; maiores do outro.

3. Recursão

Ordenar cada lado independentemente.

4. Caso base

Subvetor com zero ou um elemento.

Estratégia de pivô influencia o risco do pior caso

Comparação dos principais algoritmos de ordenação

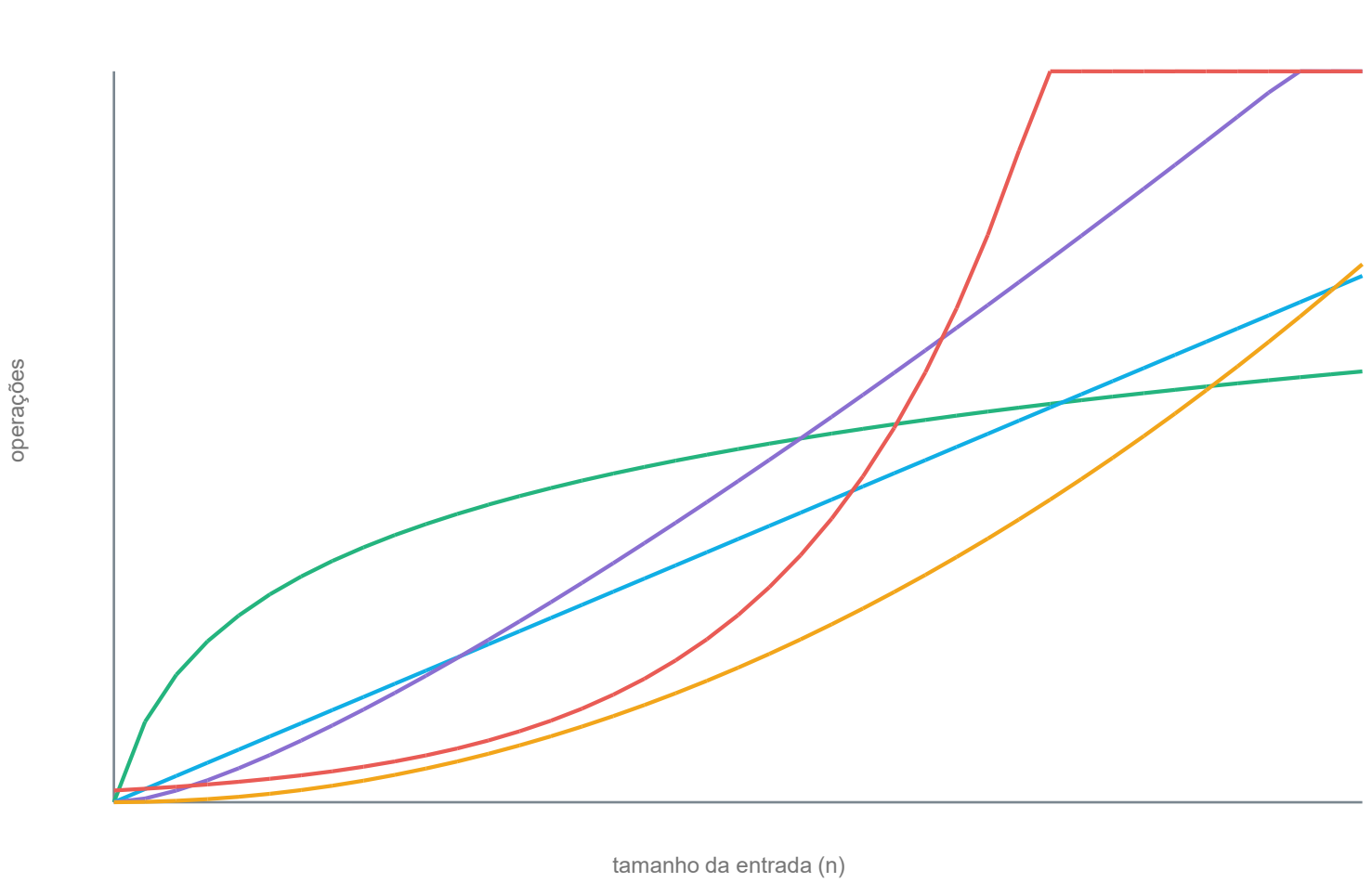
Algoritmo	Melhor	Médio	Pior	Memória extra	Estável?
Bubble otim.	O(n)	O(n²)	O(n²)	O(1)	Sim
Selection	O(n²)	O(n²)	O(n²)	O(1)	Não*
Insertion	O(n)	O(n²)	O(n²)	O(1)	Sim
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Sim
Quick	O(n log n)	O(n log n)	O(n²)	pilha recursiva	Não*

* Considerando implementações clássicas.

Não existe “o melhor sort” em qualquer situação.

Big-O: como as curvas se separam quando n cresce

A diferença entre O(n²) e O(n log n) explode em entradas grandes.



O(log n)

O(n)

O(n log n)

O(n²)

O(2^n)

Leitura da curva

Big-O não mede segundos: descreve como o trabalho cresce com n.

Ref. conceitual/visual: Wikimedia Commons – Comparison computational complexity.svg

Qual algoritmo escolher?

A decisão depende do contexto, não apenas da notação Big-O.

Poucos dados / quase ordenados

Insertion Sort costuma aproveitar bem esse cenário.

Desempenho previsível

Merge Sort mantém O(n log n) no pior caso.

Arrays grandes em memória

Quick Sort bem implementado costuma ter boa localidade e baixo overhead.

Minimizar trocas

Selection Sort faz poucas trocas, apesar de $O(n^2)$ comparações.

Didática / visualização

Bubble Sort é simples para visualizar trocas locais.

Bibliotecas reais

Muitas usam estratégias híbridas conforme tamanho e padrão da entrada.

Pergunta correta: “qual custo importa para este problema?”

Experimento de desempenho em C

Comparar teoria (Big-O) com tempo medido no computador.

```
clock_t inicio = clock();

quickSort(vetor, 0, n - 1);

clock_t fim = clock();

double tempo =
    (double)(fim - inicio) / CLOCKS_PER_SEC;

printf("%.6f s\n", tempo);
```

n	Bubble	Selection	Insertion	Merge	Quick
1.000					
5.000					
10.000					
50.000					
100.000					

Use a MESMA entrada para todos os algoritmos. Copie o vetor antes de cada teste.

Mesmo n, comportamentos diferentes

Teste entradas aleatórias, ordenadas e invertidas.

ALEATÓRIA

8	2	9	1	5	3	7

ORDENADA

1	2	3	4	5	6	7

INVERTIDA

7	6	5	4	3	2	1

Insertion

Pode ficar muito melhor em dados já/quase ordenados.

Quick

Escolha do pivô afeta o equilíbrio das partições.

Bubble otimizado

Pode parar cedo se não houver trocas.

Big-O não explica tudo: hardware também importa

Dois algoritmos com a mesma classe podem ter tempos reais diferentes.

CACHE

Dados próximos podem ser reaproveitados rapidamente.

RAM

Mais acessos à memória podem aumentar latência.

BRANCH

Desvios imprevisíveis podem reduzir eficiência do pipeline.

COMPILADOR

Otimizações mudam a quantidade de instruções geradas.

CPU

Frequência, núcleos e microarquitetura influenciam o tempo.

ALOCAÇÃO

malloc/free frequentes adicionam overhead e fragmentação.

Primeiro escolha um bom algoritmo; depois micro-otimize.

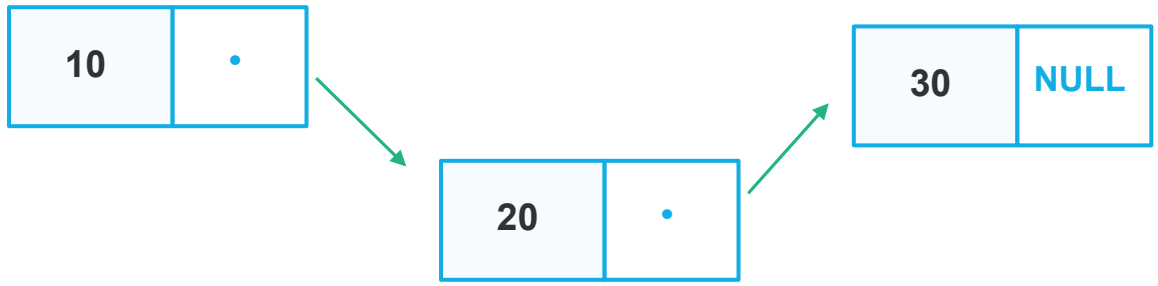
Localidade de memória: vetor × lista ligada

A organização física dos dados pode favorecer ou prejudicar o cache.

VETOR — memória contígua



LISTA — nós podem estar espalhados



L1 / L2 / L3

Caches menores e mais rápidas ficam próximas da CPU.

RAM

Maior capacidade, porém mais lenta que cache.

Impacto

Acesso sequencial tende a aproveitar melhor blocos de cache.

Ref. conceitual: Wikimedia Commons – Cache Hierarchy.png (CC BY-SA 4.0)

Otimização de software: ordem de impacto

Nem toda otimização vale o mesmo.

1	**Escolher melhor algoritmo**	Trocar $O(n^2)$ por $O(n \log n)$ pode transformar o sistema.
2	**Escolher estrutura adequada**	Fila, pilha, lista ou vetor mudam o custo das operações.
3	**Eliminar trabalho repetido**	Evitar recalcular, copiar e percorrer dados sem necessidade.
4	**Melhorar localidade e alocações**	Acessos contíguos e menos malloc/free podem ajudar.
5	**Micro-otimizar**	Só depois medir e ajustar trechos realmente críticos.

Hands-on em grupos — corrida de ordenação

Todos os grupos recebem os mesmos dados e medem o comportamento.

Grupo A

Bubble Sort

Grupo B

Selection Sort

Grupo C

Insertion Sort

Grupo D

Merge Sort

Grupo E

Quick Sort

Cada grupo deve apresentar:

• ideia do algoritmo • comparações • trocas/movimentações • tempo • Big-O • memória extra • melhor/pior caso

Mesmo vetor de entrada para todos os grupos

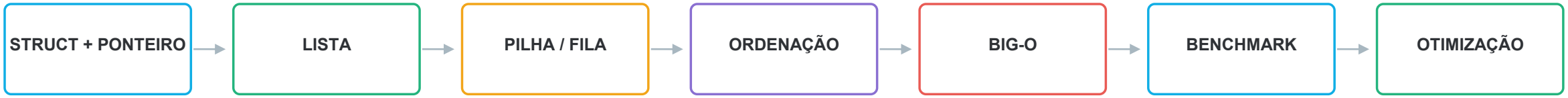
Desafio final — qual algoritmo se comporta melhor em cada cenário?

Cenário	Bubble	Selection	Insertion	Merge	Quick
Aleatório					
Ordenado					
Invertido					
Poucos elementos					
Muitos elementos					

Não preencha por “opinião”: use código, Big-O e medições.

Fechamento da aula

Estrutura + algoritmo + arquitetura = desempenho.



1. Big-O explica crescimento.
2. Benchmark mede comportamento real.
3. A estrutura de dados e o algoritmo devem ser escolhidos em conjunto.

Próxima etapa: árvores binárias e busca O(log n).

Referências visuais e conceituais

As figuras da aula foram redesenhadas em vetor a partir destas referências abertas, para manter legibilidade e compatibilidade no PowerPoint.

Lista ligada	Wikimedia Commons – Singly-linked-list.svg	https://commons.wikimedia.org/wiki/File:Singly-linked-list.svg
Pilha × fila	Wikimedia Commons – QUEUE VS STACK.svg	https://commons.wikimedia.org/wiki/File:QUEUE_VS_STACK.svg
Bubble Sort	Wikimedia Commons – Bubble sort animation.gif	https://commons.wikimedia.org/wiki/File:Bubble_sort_animation.gif
Selection Sort	Wikimedia Commons – Selection sort animation.gif	https://commons.wikimedia.org/wiki/File:Selection_sort_animation.gif
Insertion Sort	Wikimedia Commons – Insertion-sort-example.gif	https://commons.wikimedia.org/wiki/File:Insertion-sort-example.gif
Merge Sort	Wikimedia Commons – Merge sort animation.gif	https://commons.wikimedia.org/wiki/File:Merge_sort_animation.gif
Quick Sort	Wikimedia Commons – Sorting quicksort anim.gif	https://commons.wikimedia.org/wiki/File:Sorting_quicksort_anim.gif
Complexidade	Wikimedia Commons – Comparison computational complexity.svg	https://commons.wikimedia.org/wiki/File:Comparison_computational_complexity.svg
Cache	Wikimedia Commons – Cache Hierarchy.png	https://commons.wikimedia.org/wiki/File:Cache_Hierarchy.png

Licenças conforme as páginas de origem (domínio público, CC BY-SA e/ou GFDL). Consulte cada página para detalhes de atribuição.